SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment - 7.5

Name of Student : **Kesoju saikiran**
Enrollment No. : **2303A51539**
Batch No. : **22**

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.
# Bug: Mutable default argument
def add_item(item, items=[]):
items.append(item)
return items
print(add_item(1))
print(add_item(2))
Expected Output: Corrected function avoids shared list bug.

Prompt: Analyze given code where a mutable default argument causes unexpected behavior

Code:

Assignment7.py > ...

```python
#Task-1

'''def add_item(item, items=[]):
items.append(item)
return items
print(add_item(1))
print(add_item(2))'''

#Analyze given code where a mutable default argument causes unexpected behavior
#Bug: Mutable default argument
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
print(add_item(1))
print(add_item(2))
```

Output:

```
PS C:\Users\Dell\Desktop\PYTHON>

  [1]
  [2]
```

Task—2: Task: Analyze given code where floating-point comparison fails.
Use AI to correct with tolerance.
# Bug: Floating point precision issue
def check_sum():
return (0.1 + 0.2) == 0.3
print(check_sum())
Expected Output: Corrected function

Prompt: Analyze given code where floating-point comparison fails.

Code:

```
#Task-2

'''def check_sum():
return (0.1 + 0.2) == 0.3
print(check_sum())'''

#Analyze given code where floating-point comparison fails.
#Bug: Floating point precision issue
from decimal import Decimal, getcontext

def check_sum():
    getcontext().prec = 10
    return Decimal('0.1') + Decimal('0.2') == Decimal('0.3')

print(check_sum())
```

Output:

```
PS C:\Users\Dell\Desktop\PYTHON>

True
```

Task-3: Analyze given code where recursion runs infinitely due to
missing base case. Use AI to fix.
# Bug: No base case
def countdown(n):
print(n)
return countdown(n-1)
countdown(5)
Expected Output : Correct recursion with stopping condition

Prompt:
Analyze given code where recursion runs infinitely due to missing base case.

Code:

```
#Task-3

'''def countdown(n):
print(n)
return countdown(n-1)
countdown(5)'''

#Analyze given code where recursion runs infinitely due to missing base case.
#Bug: No base case.
def countdown(n):
    if n <= 0:
        print("Done")
        return
    print(n)
    return countdown(n-1)

countdown(5)
```

Output:

SR UNIVERSITY

```
PS C:\Users\Dell\Desktop\PYTHON>

5
4
3
2
1
Done
```

Task—4 :

Analyze given code where a missing dictionary key causes error. Use AI to fix it.

# Bug: Accessing non-existing key
def get_value():
data = {"a": 1, "b": 2}
return data["c"]
print(get_value())

Expected Output: Corrected with .get() or error handling.

Prompt: Analyze given code where a missing dictionary key causes error.

Code:

```
#Task-4

'''def get_value():
data = {"a": 1, "b": 2}
return data["c"]
print(get_value())'''

#Analyze given code where a missing dictionary key causes error.
#Bug: Accessing non-existing key.

def get_value():
    data = {"a": 1, "b": 2}
    return data.get("c")
print(get_value())
```

Output:

```
PS C:\Users\Dell\Desktop\PYTHON>

None
```

Task—5 :

Analyze given code where loop never ends. Use AI to detect and fix it.

```
# Bug: Infinite loop
def loop_example():
i = 0
while i < 5:
print(i)
```

Expected Output: Corrected loop increments i.

Prompt: Analyze given code where loop never ends.

Code:

```
#Task-5

'''def loop_example():
i = 0
while i < 5:
print(i)'''
#Analyze given code where loop never ends.
#Bug: Infinite loop.

def loop_example():
    i = 0
    while i < 5:
        print(i)
        i += 1

if __name__ == "__main__":
    loop_example()
```

Output:

```
PS C:\Users\Dell\Desktop\PYTHON>

0
1
2
3
4
```

Task—6:

Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

Prompt: Analyze given code where tuple unpacking fails.

CODE:

```
#Task-6

'''a, b = (1, 2, 3)'''

#Analyze given code where tuple unpacking fails.
#Bug: Mismatched number of values to unpack.

a, b, _ = 1, 2, 3

if __name__ == "__main__":
    print(a, b)
```

Output:

```
PS C:\Users\Dell\Desktop\PYTHON>

1 2
```

Task—7 :

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it. # Bug: Mixed indentation def func(): x = 5 y = 10 return x+y

Expected Output : Consistent indentation applied.

Prompt:

Analyze given code where mixed indentation breaks execution.

Code:

```
#Task-7

'''def func():
x = 5
y = 10
return x+y '''

#Analyze given code where mixed indentation breaks execution.
#Bug: Mixed indentation (tabs and spaces).


def func() -> int:
    x = 5
    y = 10
    return x + y

if __name__ == "__main__":
    print(func())
```

Output:

```
PS C:\Users\Dell\Desktop\PYTHON>

15
```

Task—8:

Analyze given code with incorrect import. Use AI to fix.
# Bug: Wrong import
import maths
print(maths.sqrt(16))
Expected Output: Corrected to import math

Prompt: Analyze given code with incorrect import.

Code:

```
#Task-8
'''
import maths
print(maths.sqrt(16))
'''
#Analyze given code with incorrect import.
#Bug: Incorrect import statement.

import math

if __name__ == "__main__":
    print(math.sqrt(16))
```

Output:

```
PS C:\Users\Dell\Desktop\PYTHON>

4.0
```